SERV RISC-V + CXXRTL + Zephyr RTOS

Definitive Engineering Reference

Browser-Based Cycle-Accurate Simulation via Yosys CXXRTL → WebAssembly

---

Everything you need to never suffer what we suffered.

Reverse-engineered from RTL. Confirmed from silicon. Documented in blood.

Table of Contents

Table of Contents ... 2
Part 1: System Architecture — The Complete Picture ... 4
1.1 The Full Stack ... 4
1.2 SERV CPU Architecture — Critical Facts ... 4
1.2.1 Bit-Serial Execution ... 4
1.2.2 Instruction Timing ... 5
1.2.3 Register File Layout in CXXRTL ... 5
1.3 The RF Write Pipeline — THE Most Important Section ... 6
Part 2: Memory Map — Confirmed from Devicetree ... 8
2.1 NS16550 UART — Exact Bus Transactions ... 9
2.2 UART Initialization Sequence ... 9
2.3 CLINT Timer ... 10
Part 3: CXXRTL Signal Reading — Critical Pitfalls ... 10
3.1 Signal Types in CXXRTL ... 10
3.2 Key Internal Signals — Accessor Reference ... 10
3.3 How to Find Signal Names from sim.cpp ... 11
Part 4: Boot Sequence — From Reset to Hello World ... 12
4.1 The ROM Trampoline Problem ... 12
4.2 Boot Trace — Confirmed Addresses (hello_world binary) ... 12
4.3 mtvec Write — The Critical Boot Instruction ... 13
Part 5: Zephyr Build — Zero OS Modification ... 13
5.1 Target Binary Specification ... 13
5.2 The wfi Problem ... 13
5.2.1 Application Kconfig Injection ... 14
5.2.2 Custom Idle Implementation ... 14
5.2.3 CMakeLists.txt Update ... 14
5.3 The Complete Build Script ... 15
5.4 The Single Remaining wfi ... 16
Part 6: Driver Architecture — The Complete C++ Driver ... 16
6.1 Step Function Structure ... 16
6.2 dbus Address Decode ... 17
6.3 UART State Machine ... 17
6.4 Binary Loading ... 18
Part 7: Diagnostic System — How to See Everything ... 18
7.1 Required C++ Exports ... 18

7.2 The Diagnostic Button Snapshot 19
7.3 diagStep() — The Production Version 20
7.4 The Unknown MMIO Log 21
Part 8: Debugging Playbook — Symptom to Root Cause 21
8.1 PC Stuck Below 0x80000000 21
8.2 mtvec Wrong After Boot 22
8.3 Trap Storm After Hello World 22
8.4 Stall at arch_system_halt 22
8.5 UART Log Empty at Cycle 100000 22
8.6 Register Reads Always Zero 22
8.7 Unknown MMIO Accesses in Log 22
8.8 Derailment to Small Address 23
8.9 cnt_en and wcnt Always Read 0 23
Part 9: Lessons — How to Be 10x More Efficient 23
9.1 What Wasted the Most Time 23
9.1.1 Not Reading the RTL Files First 23
9.1.2 Not Having Internal Signal Probes From Day One 24
9.1.3 Confusing CXXRTL Outline vs Registered Signals 24
9.1.4 Fixing Symptoms Instead of Causes 24
9.2 The Correct Debugging Protocol 24
9.3 The Minimal Diagnostic Set for Any New CXXRTL Driver 25
9.4 Things That Were Genuinely Not Our Fault 25
9.4.1 The 1-Cycle ibus_ack Timing Requirement 25
9.4.2 CXXRTL Outline Signal Staleness 25
9.4.3 Zephyr ARCH_HAS_CUSTOM_CPU_IDLE Not Settable in prj.conf 25
9.4.4 wfi in SMP Secondary Core Loop 26
Part 10: Quick Reference 26
10.1 Checklist — Before Running Any Binary 26
10.2 Checklist — After First Boot 26
10.3 Signal Name Cheat Sheet 27
10.4 Timing Quick Reference 27
Appendix: Files Modified in Zephyr Tree 28

Part 1: System Architecture — The Complete Picture

This document describes every layer of a browser-based cycle-accurate RISC-V simulation stack. Understanding all layers simultaneously is the single most important thing for effective debugging. A bug in any one layer masquerades as a bug in all adjacent layers.

1.1 The Full Stack

From bottom to top:

Layer	Component / Description
Hardware (RTL)	SERV — olofk/serv — World's smallest RV32I CPU. Bit-serial. 1-bit datapath. W=1.
Synthesis	Yosys — Synthesizes SERV Verilog to C++ via CXXRTL backend
C++ Driver	Hand-written driver wrapping CXXRTL sim. Handles ibus/dbus, UART, CLINT, SRAM
WebAssembly	Clang (wasm32 target) compiles C++ driver to .wasm
JavaScript	Instantiates .wasm, drives clock loop, handles UI, diagnostics
RTOS Binary	Zephyr OS — qemu_riscv32 board — pure rv32i_zicsr_zifencei
Host Platform	Browser (W3Schools TryIt / any modern browser)

1.2 SERV CPU Architecture — Critical Facts

SERV is not a conventional CPU. Every non-obvious behavior stems from its bit-serial design. Read this section before touching anything.

1.2.1 Bit-Serial Execution

SERV processes registers 1 bit per clock cycle. A 32-bit addition takes 32 clock cycles. This is not a pipeline — it is literally shifting bits through a 1-bit ALU.

Parameter	Value and Meaning
W	1 — datapath width in bits. Everything is 1 bit wide.
RF_WIDTH	2 — register file storage width (W*2). 2-bit RAM words.
depth	576 — RF RAM entries. 36 registers × 32 bits / 2 bits-per-word = 576

ratio	2 — RF_WIDTH / W = 2. Two RF bits stored per RAM address.
CMSB	4 — counter MSB index. log2(W) subtracted from 4.

1.2.2 Instruction Timing

Instruction Type	Cycles — Notes
Single-stage (addi, add, CSR, ecall, mret)	35 — Standard execution
Two-stage (shifts, branches, jumps)	67 — Two phases
Memory (sw/sh/sb/lw/lh/lb)	68 — Load/store with dbus
Trap entry (ibus_ack → handler fetch)	35 — From ecall/timer IRQ
Misaligned trap entry	69 — Extra phase for alignment

1.2.3 Register File Layout in CXXRTL

SERV stores all 36 registers (32 GPR + 4 CSR) as 2-bit chunks in a flat RAM array. The CXXRTL signal is:

```
dut.memory_p_cpu_2e_rf__ram_2e_memory.data[i].data[0]  // 2-bit value at index i
```

Register	Index in RF RAM
x0 (zero)	0–15
x1–x31	16–511
x32 (mscratch)	512–527
x33 (mtvec)	528–543
x34 (mepc)	544–559
x35 (mtval)	560–575

To read register r from JavaScript:

```
function get_rf_reg(r) {
    let val = 0;
    const base = r * 16;
    for(let i = 0; i < 16; i++) {
        val |= (cpu.get_rf_word(base + i) << (i * 2));
    }
```

```
    return val >>> 0;
}
```

1.3 The RF Write Pipeline — THE Most Important Section

> ⚠ **CRITICAL:** This section describes the single root cause of the most difficult bug in this entire project. Read it completely before writing any driver code.

The RF write pipeline in serv_rf_ram_if.v operates as follows:

rcnt — The RF Interface Counter

rcnt is a 5-bit counter internal to serv_rf_ram_if. It is completely separate from the CPU execution counter (cnt_lsb/cnt_en). It counts independently from 0 upward every cycle, and resets to 0 when i_rreq fires (on ibus_ack) or to 2 when i_wreq fires.

```
// From serv_rf_ram_if.v line 100:
assign wcnt = rcnt - 4;

// rcnt reset rule (line 164-165):
if (i_rreq | i_wreq)
    rcnt <= {3'b000, i_wreq, 1'b0};
// i_rreq=1, i_wreq=0 → rcnt resets to 0b00000 = 0
// i_wreq=1           → rcnt resets to 0b00010 = 2
```

Write Trigger Timing

wtrig1 = wcnt[0] fires at every odd wcnt value. Each trigger writes 2 bits (RF_WIDTH=2) to the RAM. To write all 32 bits of a register requires 16 triggers at wcnt = 1, 3, 5, 7, ... 31.

wen1_r (the CSR port write enable latch) is re-evaluated at each wtrig1 pulse:

```
// wen1_r.next is latched when wcnt[0]=1:
wen1_r.next = (wcnt[0] ?
    AND(rf_if.i_cnt_en, OR(i_trap, i_csr_en))
    : wen1_r.curr);
```

The Critical Timing Problem

> ● **ROOT CAUSE:** If ibus_ack is asserted on the FIRST cycle ibus_cyc goes high, rcnt resets to 0 immediately. The first wtrig1 fires at rcnt=5 (wcnt=1), but wen1_r.curr at that

moment is still 0 (from the PREVIOUS instruction). The latch updates wen1_r.next=1 AT the same rising edge — but the write uses .curr. So chunk 0 is never written.

Additionally, the final chunk (chunk 15, bits 31:30) requires wcnt=31 which means rcnt=35. Since rcnt is 5 bits (max 31), it wraps to 3. The write at rcnt=3/wcnt=31 fires at the SAME cycle as ibus_ack for the next instruction, which resets rcnt and aborts the write.

The Fix — Delay ibus_ack by One Cycle

Asserting ibus_ack on the SECOND cycle that ibus_cyc is high shifts all write triggers by one cycle, giving:

- wen1_r=1 before the first wtrig1 fires → chunk 0 written correctly
- The rcnt=3 write for chunk 15 completes before next ibus_ack resets rcnt

```
// CORRECT driver ibus handling:
uint32_t icyc = dut.p_ibus__cyc.data[0];
if(icyc && !prev_icyc) {
    // First cycle: present data, NO ACK
    current_pc = dut.p_ibus__adr.data[0];
    if(current_pc >= 0x80000000u)
        dut.p_ibus__rdt.data[0] = sram[(current_pc>>2)&0xFFFF];
    else
        dut.p_ibus__rdt.data[0] = rom[(current_pc>>2)&0xFFFF];
    // Do NOT assert ibus_ack here
}
if(icyc && prev_icyc) {
    // Second cycle: NOW ack
    dut.p_ibus__ack.data[0] = 1;
}
prev_icyc = icyc;
```

How We Found This

The diagnosis required exporting 6 internal CXXRTL signals and running a cycle-by-cycle trace from cycle 243 to 320:

```
// Required exports (signal names from synthesized sim.cpp):
dut.p_cpu_2e_rf__ram__if_2e_rcnt.curr.data[0]      // rcnt
dut.p_cpu_2e_rf__ram__if_2e_wcnt.data[0]           // wcnt (outline — stale!)
dut.p_cpu_2e_rf__ram__if_2e_wen1__r.curr.data[0]   // wen1_r
dut.p_cpu_2e_rf__ram__if_2e_wdata1__r.curr.data[0] // wdata1_r
dut.p_cpu_2e_rf__ram__if_2e_rtrig1.curr.data[0]    // rtrig1
dut.p_cpu_2e_cpu_2e_state_2e_gen__cnt__w__eq__1_2e_cnt__lsb.curr.data[0] // cnt_lsb
```

> ⚠ **WARNING:** wcnt and cnt_en are /*outline*/ signals in CXXRTL. Reading them with .data[0] after step() returns STALE values. They always read as 0. Only use .curr on registered wire<N> signals. Use cnt_lsb (registered) as a proxy for cnt_en.

The smoking gun in the trace data (cycle-by-cycle, cycles 243-286):

`cycle=250`	rcnt=0, rreq_r=1 — ibus_ack fired, rcnt reset
`cycle=251`	rcnt=1, rgnt=1 — rgnt latched one cycle after rreq_r
`cycle=252`	rcnt=2, cnt_lsb=1 — CPU counter starts (cnt_en=1 from here)
`cycle=254`	rcnt=4, wen1_r=1 — latched at wcnt=0 (rcnt=4)
`cycle=258`	rcnt=8, mtvec=0xc — first write (chunk 1, bits 3:2)
`cycle=260`	rcnt=10, mtvec=0x1c — second write (chunk 2, bits 5:4)
`cycle=284`	rcnt=2, wdata1_r=4 — bit 31 of t0 arrives in shift register
`cycle=286`	rcnt=0, mtvec=0x8000001c — chunk 15 written! ✓

Part 2: Memory Map — Confirmed from Devicetree

The memory map is derived from the Zephyr qemu_riscv32 devicetree. Every address is confirmed from devicetree_generated.h and the ns16550 driver source.

Address Range	Device
`0x00000000–0x0000FFFF`	ROM / Trampoline
`0x02000000`	mtime lo (alias)
`0x02000004`	mtime hi (alias)
`0x0200BFF8`	CLINT mtime lo
`0x0200BFFC`	CLINT mtime hi
`0x02004000`	CLINT mtimecmp lo
`0x02004004`	CLINT mtimecmp hi
`0x0C000000+`	PLIC
`0x10000000`	NS16550 UART base
`0x80000000–0x8000FFFF`	SRAM (64KB)

2.1 NS16550 UART — Exact Bus Transactions

reg-shift=0 → reg_interval=1. CONFIG_UART_NS16550_ACCESS_WORD_ONLY=n. All accesses are byte-wide via sys_write8/sys_read8.

Register	Byte Addr
THR/RBR	0x10000000
IER	0x10000001
FCR	0x10000002
LCR	0x10000003
LSR	0x10000005

The driver must return 0x60 from LSR reads to indicate TX ready. The byte must be in the correct lane position based on sel:

```
// CORRECT byte lane response:
uint32_t shift = (sel&2)?8 : (sel&4)?16 : (sel&8)?24 : 0;
dut.p_dbus__rdt.data[0] = (value & 0xFF) << shift;

// For LSR (reg=5):
uint32_t lsr = 0x60 | (uart_rx != -1 ? 1 : 0);
dut.p_dbus__rdt.data[0] = (lsr & 0xFF) << shift;
```

2.2 UART Initialization Sequence

Zephyr's ns16550 driver performs these bus transactions in order on startup:

Transaction	Register
0	LSR
1	FCR
2	LCR
3	MCR
4	FCR
5	FCR
6	LSR
7	IER

2.3 CLINT Timer

Zephyr uses the standard RISC-V CLINT timer. mtime increments once per CPU cycle in the driver (tick_mtime called every step()). mtimecmp is written with a safe-write sequence:

```
// Zephyr timer init sequence:
// 1. Write mtimecmp_hi = 0xFFFFFFFF (prevent spurious IRQ)
// 2. Write mtimecmp_lo = actual_value
// 3. Write mtimecmp_hi = actual_value_hi

// Driver IRQ logic:
uint64_t mt  = ((uint64_t)mtime_hi << 32) | mtime_lo;
uint64_t cmp = ((uint64_t)mtimecmp_hi << 32) | mtimecmp_lo;
dut.p_timer__irq.data[0] = (mt >= cmp) ? 1 : 0;
```

Part 3: CXXRTL Signal Reading — Critical Pitfalls

3.1 Signal Types in CXXRTL

CXXRTL generates two fundamentally different signal types. Confusing them wastes days.

Type	Declaration
Registered (wire<N>)	wire<1> p_signal;
Combinatorial (outline)	/*outline*/ value<N> p_signal;

🔴 **TRAP:** Outline signals (wcnt, cnt_en, o_wen1, o_wen0) all read as 0 after step() because CXXRTL does not re-evaluate them unless something downstream forces evaluation. Do not use outline signals for diagnosis. Use registered proxy signals instead.

3.2 Key Internal Signals — Accessor Reference

Signal names are generated by Yosys from the RTL hierarchy. The naming scheme is: module path with '.' replaced by '_2e_' and special chars encoded.

```
// REGISTERED signals — safe to read after step():
dut.p_cpu_2e_rf__ram__if_2e_rcnt.curr.data[0]
    // rcnt: 5-bit RF interface counter. wire<5>

dut.p_cpu_2e_rf__ram__if_2e_wen1__r.curr.data[0]
    // wen1_r: CSR write enable latch. wire<1>

dut.p_cpu_2e_rf__ram__if_2e_wdata1__r.curr.data[0]
    // wdata1_r: CSR write data shift register. wire<3>

dut.p_cpu_2e_rf__ram__if_2e_rtrig1.curr.data[0]
    // rtrig1: Read trigger. wire<1>

dut.p_cpu_2e_rf__ram__if_2e_rgnt.curr.data[0]
    // rgnt: RF ready grant. wire<1>

dut.p_cpu_2e_rf__ram__if_2e_rreq__r.curr.data[0]
    // rreq_r: RF read request registered. wire<1>

dut.p_cpu_2e_cpu_2e_state_2e_gen__cnt__w__eq__1_2e_cnt__lsb.curr.data[0]
    // cnt_lsb: 4-bit CPU counter shift register. wire<4>
    // cnt_en = (cnt_lsb != 0). Use this as proxy for cnt_en.

// OUTLINE signals — DO NOT use for debugging (always stale):
// p_cpu_2e_rf__ram__if_2e_wcnt   — always reads 0
// p_cpu_2e_cpu_2e_state_2e_o__cnt__en — always reads 0
// p_cpu_2e_cpu_2e_rf__if_2e_o__wen1  — always reads 0
```

3.3 How to Find Signal Names from sim.cpp

When you need to probe a new internal signal, use this method in the browser after synthesis:

```
// In bootSoC(), after sim.cpp is generated:
const lines = simCpp.split('\n');

// Find all signals matching a pattern:
console.log(simCpp.match(/p_cpu[^;\s]*YOUR_SIGNAL_NAME[^;\s]*/g));

// Find the type of a specific signal:
console.log(lines.filter(l =>
    l.includes('YOUR_SIGNAL') &&
    (l.includes('wire') || l.includes('value<'))
).slice(0, 5));
```

Part 4: Boot Sequence — From Reset to Hello World

4.1 The ROM Trampoline Problem

SERV resets to PC=0x00000000. The Zephyr binary lives at 0x80000000. ROM at 0x00000000 is all zeros (nop sled). Without a trampoline, SERV executes nops forever.

Symptom: PC increases linearly at rate 1 instruction per 35 cycles. Each heartbeat shows PC approximately 0xE5C8 higher than the last. This is the nop sled signature.

Fix: Plant a 2-instruction trampoline in ROM after loading the binary:

```
// auipc t0, 0x80000  →  t0 = PC(0) + 0x80000000 = 0x80000000
cpu.load_rom(0, 0x80000297);

// jalr x0, t0, 0    →  PC = t0 = 0x80000000
cpu.load_rom(1, 0x00028067);

// Encoding verification:
// auipc t0, 0x80000: (0x80000 << 12) | (5 << 7) | 0x17 = 0x80000297
// jalr x0, t0, 0:   (0 << 20) | (5 << 15) | (0 << 12) | (0 << 7) | 0x67 =
0x00028067
```

4.2 Boot Trace — Confirmed Addresses (hello_world binary)

Cycle	PC
105	0x80000000
177	0x80000008
243	0x80000010
285	0x80000014
353	0x80000018
421	0x80001158
712916	0x80000238
755118	—

4.3 mtvec Write — The Critical Boot Instruction

At cycle 243, csrw mtvec executes. With the 1-cycle ibus_ack delay fix:

- t0=0x8000001c going in (confirmed from [PRE-CSRW] log)
- mtvec=0x8000001c coming out (confirmed from [POST-CSRW] log at cycle 285)
- Without the fix: mtvec=0x1c (only 5 bits written)

> 📌 **INVARIANT:** If mtvec != 0x8000001c after cycle 285, the ibus_ack delay fix was not applied correctly. This is the single most important diagnostic check on every boot.

Part 5: Zephyr Build — Zero OS Modification

5.1 Target Binary Specification

Property	Value
ISA	rv32i2p1_zicsr2p0_zifencei2p0
No M extension	No multiply/divide instructions
No C extension	No compressed 16-bit instructions
No F/D extension	No floating point
Entry point	0x80000000
Binary end	~0x800072E0 (≈29KB used of 64KB SRAM)
Stack top	0x800072E0 (z_mapped_end)
z_interrupt_stacks	0x800064E0, size 0xE00
z_main_stack	0x80006EE0

5.2 The wfi Problem

Zephyr's arch_cpu_idle() contains a wfi instruction. SERV does not implement wfi (it is not part of RV32I base). Executing wfi causes an illegal instruction trap (mcause=3) every time the idle thread runs, creating an infinite trap storm that prevents the scheduler from running.

The elegant fix uses Zephyr's own override mechanism without touching any OS source:

5.2.1 Application Kconfig Injection

Create samples/hello_world/Kconfig to inject the override at the Kconfig root level:

```
mainmenu "SERV Application"
source "Kconfig.zephyr"

config SERV_CUSTOM_IDLE
        def_bool y
        select ARCH_HAS_CUSTOM_CPU_IDLE
        select ARCH_HAS_CUSTOM_CPU_ATOMIC_IDLE
```

WHY THIS WORKS: ARCH_HAS_CUSTOM_CPU_IDLE has no prompt so prj.conf cannot set it directly. But select bypasses the prompt restriction. By defining our own Kconfig with a mainmenu (hijacking the config root), we can select any hidden symbol. This is the exact mechanism used by Intel ADSP, Nordic VPR, and WCH CH32V SoCs in the Zephyr tree.

5.2.2 Custom Idle Implementation

Add src/serv_idle.c to the application (separate from main.c):

```
#include <zephyr/irq.h>

/* SERV is a minimal RV32I core. wfi is not implemented.
 * Idle by unlocking interrupts and returning immediately. */
void arch_cpu_idle(void) {
        irq_unlock(MSTATUS_IEN);
}

void arch_cpu_atomic_idle(unsigned int key) {
        irq_unlock(key);
}
```

WHY irq_unlock IS MANDATORY: Zephyr calls arch_cpu_idle with interrupts LOCKED. The architectural contract is that the idle function must re-enable interrupts before returning. If you just return without irq_unlock, interrupts stay locked forever, the timer never fires, and the scheduler never wakes up. The system hangs after Hello World.

5.2.3 CMakeLists.txt Update

Register the new file with CMake:

```
# Append to samples/hello_world/CMakeLists.txt:
```

```
target_sources(app PRIVATE src/serv_idle.c)
```

5.3 The Complete Build Script

Idempotent — works from scratch or iterative state:

```
#!/bin/bash
set -e

pip3 install west pyelftools 2>/dev/null || true
pip3 install -r scripts/requirements.txt 2>/dev/null || true
west init -l . 2>/dev/null || true

# Config
cat > /tmp/serv_prj.conf << 'EOF'
CONFIG_BUILD_OUTPUT_BIN=y
CONFIG_RISCV_PMP=n
CONFIG_FPU=n
EOF

# Device tree overlay — strip all extensions except i/zicsr/zifencei
cat > /tmp/serv2.overlay << 'EOF'
/ {
    cpus {
        cpu@0 { riscv,isa-extensions = "i", "zicsr", "zifencei"; };
        cpu@1 { riscv,isa-extensions = "i", "zicsr", "zifencei"; };
        // ... repeat for cpu@2 through cpu@7
    };
};
EOF

# Kconfig injection (idempotent)
cat > /workspaces/zephyr/samples/hello_world/Kconfig << 'EOF'
mainmenu "SERV Application"
source "Kconfig.zephyr"
config SERV_CUSTOM_IDLE
        def_bool y
        select ARCH_HAS_CUSTOM_CPU_IDLE
        select ARCH_HAS_CUSTOM_CPU_ATOMIC_IDLE
EOF

# Custom idle (idempotent)
cat > samples/hello_world/src/serv_idle.c << 'EOF'
#include <zephyr/irq.h>
void arch_cpu_idle(void) { irq_unlock(MSTATUS_IEN); }
void arch_cpu_atomic_idle(unsigned int key) { irq_unlock(key); }
EOF
```

```
if ! grep -q 'serv_idle' samples/hello_world/CMakeLists.txt; then
    echo 'target_sources(app PRIVATE src/serv_idle.c)' >> \
        samples/hello_world/CMakeLists.txt
fi

rm -rf build/
west build -b qemu_riscv32 samples/hello_world \
  -- -DEXTRA_CONF_FILE=/tmp/serv_prj.conf \
     -DDTC_OVERLAY_FILE=/tmp/serv2.overlay

# Verify
echo '=== ISA ===' && riscv64-zephyr-elf-readelf -A build/zephyr/zephyr.elf | grep
tag
echo '=== wfi (expect: 1 unreachable SMP only) ===' && \
    riscv64-zephyr-elf-objdump -d build/zephyr/zephyr.elf | grep -B2 wfi
ls -la build/zephyr/zephyr.bin
```

5.4 The Single Remaining wfi

After the fix, one wfi remains in the binary at loop_unconfigured_cores (reset.S:126). This is the SMP secondary core idle loop. Since SERV is always hart 0 and never boots secondary cores, this code is unreachable. It is harmless.

Part 6: Driver Architecture — The Complete C++ Driver

6.1 Step Function Structure

The step() function runs one complete clock cycle (falling edge + rising edge). Order matters critically.

```
void step() {
    tick_mtime();              // Increment timer BEFORE clock edge

    dut.p_clk.data[0] = 0;
    dut.step();                // Falling edge — combinatorial settle

    // ibus handling — MUST check on falling edge
    uint32_t icyc = dut.p_ibus__cyc.data[0];
    if(icyc && !prev_icyc) {  // Rising edge of ibus_cyc only
        current_pc = dut.p_ibus__adr.data[0];
        // Present data but DO NOT ACK
        dut.p_ibus__rdt.data[0] = (current_pc >= 0x80000000u)
```

```
            ? sram[(current_pc>>2)&0xFFFF]
            : rom[(current_pc>>2)&0xFFFF];
    }
    if(icyc && prev_icyc) {   // Second cycle — NOW ack
        dut.p_ibus__ack.data[0] = 1;
    }
    prev_icyc = icyc;

    // dbus handling
    uint32_t dcyc = dut.p_dbus__cyc.data[0];
    if(dcyc) {
        // ... address decode and respond ...
        dut.p_dbus__ack.data[0] = 1;
    } else {
        dut.p_dbus__ack.data[0] = 0;
    }
    prev_dcyc = dcyc;

    dut.p_clk.data[0] = 1;
    dut.step();                // Rising edge — registers update

    dut.p_ibus__ack.data[0] = 0;  // Clear ack after rising edge
    dut.p_dbus__ack.data[0] = 0;
    sim_cycle++;
}
```

6.2 dbus Address Decode

Critical: SERV generates byte-aligned addresses with sel indicating the active byte lane. Always extract the byte from the correct lane.

```
// Byte offset from sel:
uint32_t byte_offset = 0;
if (sel & 2) byte_offset = 1;
else if (sel & 4) byte_offset = 2;
else if (sel & 8) byte_offset = 3;

// Extract byte from write data:
uint32_t byte_val = (ddat >> (byte_offset * 8)) & 0xFF;

// Return byte in correct lane for reads:
uint32_t shift = (sel&2)?8 : (sel&4)?16 : (sel&8)?24 : 0;
dut.p_dbus__rdt.data[0] = (value & 0xFF) << shift;
```

6.3 UART State Machine

The UART must track LCR (particularly bit 7 — DLAB) to distinguish THR/RBR from divisor register access:

```
// UART handler — called when dadr is in 0x10000000-0x1000000F range:
uint32_t reg = (dadr & 0xF) + byte_offset;

if(dwe && !prev_dcyc) {
    if(reg == 3) uart_lcr = byte_val;             // LCR
    else if(reg == 0 && !(uart_lcr & 0x80)) {     // THR (DLAB=0)
        if(tx_idx < 65535) uart_tx[tx_idx++] = (char)byte_val;
    }
    // All other writes (IER, FCR, MCR) silently ignored
} else if(!dwe) {
    if(reg == 5) {  // LSR — ALWAYS return TX ready
        dut.p_dbus__rdt.data[0] = (0x60 | (uart_rx!=-1?1:0)) << shift;
    }
    else if(reg == 0 && !(uart_lcr & 0x80)) {  // RBR
        if(!prev_dcyc) { latched_rx = uart_rx; uart_rx = -1; }
        dut.p_dbus__rdt.data[0] = (latched_rx & 0xFF) << shift;
    }
    else dut.p_dbus__rdt.data[0] = 0;
}
```

6.4 Binary Loading

```
function loadBinaryIntoSRAM(arrayBuffer) {
    const bytes = new Uint8Array(arrayBuffer);
    const count = Math.floor(bytes.length / 4);
    cpu.init();
    for(let i = 0; i < count; i++) {
        const w = (bytes[i*4]) | (bytes[i*4+1]<<8) |
                  (bytes[i*4+2]<<16) | (bytes[i*4+3]<<24);
        cpu.load_sram(i, w >>> 0);
    }
    // ROM trampoline: jump from reset addr 0x0 to binary at 0x80000000
    cpu.load_rom(0, 0x80000297); // auipc t0, 0x80000
    cpu.load_rom(1, 0x00028067); // jalr  x0, t0, 0
}
```

Part 7: Diagnostic System — How to See Everything

7.1 Required C++ Exports

Minimum exports needed for definitive diagnosis of any boot failure:

```
// Basic state
get_pc()           → current_pc (set from ibus_adr on ibus_cyc rising edge)
get_reg(r)         → GPR value reconstructed from RF RAM chunks
get_rf_word(i)     → raw 2-bit chunk at RF RAM index i
get_rf_reg(r)      → 32-bit register value from chunks (same as get_reg but
includes CSRs)
get_mtvec()        → get_rf_reg(33)
set_mtvec(val)     → write all 16 chunks of RF[33] directly
set_rf_reg(r, val) → write any register directly

// UART
read_tx_idx()      → number of bytes transmitted
read_tx_char(i)    → transmitted byte at index i
send_rx(c)         → inject byte into UART RX

// Timer
get_mtime_lo/hi()  → current mtime value
get_irq_count()    → number of timer IRQs fired
get_first_irq_cycle() → mtime when first IRQ fired

// Diagnostics
get_mtime_read_count() → how many times Zephyr read mtime (confirms timer driver
init)
get_uart_log_count/entry() → first 20 UART bus transactions (confirms ns16550 init)
get_unknown_count/addr/we/val/cycle() → all unhandled MMIO accesses
get_mcause()       → last known mcause value

// Internal RF pipeline (for CSR write debugging)
get_rcnt()         → dut.p_cpu_2e_rf__ram__if_2e_rcnt.curr.data[0]
get_wen1_r()       → dut.p_cpu_2e_rf__ram__if_2e_wen1__r.curr.data[0]
get_wdata1_r()     → dut.p_cpu_2e_rf__ram__if_2e_wdata1__r.curr.data[0]
get_rtrig1()       → dut.p_cpu_2e_rf__ram__if_2e_rtrig1.curr.data[0]
get_cnt_lsb()      →
dut.p_cpu_2e_cpu_2e_state_2e_gen__cnt__w__eq__1_2e_cnt__lsb.curr.data[0]
get_rgnt()         → dut.p_cpu_2e_rf__ram__if_2e_rgnt.curr.data[0]
get_rreq_r()       → dut.p_cpu_2e_rf__ram__if_2e_rreq__r.curr.data[0]
```

7.2 The Diagnostic Button Snapshot

A button in the UI that dumps the full machine state to the terminal on demand. The most important fields:

- mtvec value and binary representation (check bit 31 is set)
- All 32 registers
- Stack dump around SP
- BSS region sanity check
- UART transaction log (first 20)

- Unknown MMIO access log
- Boot trace (all unique PCs visited in 0x80000000-0x80001200)
- Timer state (mtime, mtimecmp, irq_count)

7.3 diagStep() — The Production Version

Generic version that works for any binary (no hardcoded PC addresses):

```
function diagStep() {
    const prePc = cpu.get_pc() >>> 0;
    cpu.step();
    DIAG.cycleCount++;
    const postPc = cpu.get_pc() >>> 0;

    // Stall detection
    if(postPc === prePc) {
        if(++DIAG.stallCount === 10000)
            console.error(`[STALL] PC=0x${postPc.toString(16)}`);
    } else { DIAG.stallCount = 0; }

    // PC history ring buffer
    DIAG.pcHistory.push(prePc);
    if(DIAG.pcHistory.length > 8) DIAG.pcHistory.shift();

    // Derailment — PC left SRAM into unmapped space
    if(prePc >= 0x80000000 && postPc < 0x80000000 && postPc > 0x00000008) {
        console.error(`[DERAILMENT] ${prePc.toString(16)} →
${postPc.toString(16)}`);
        for(let r=0;r<32;r++) console.error(`
${ABI[r]}=${(cpu.get_reg(r)>>>0).toString(16)}`);
    }

    // First SRAM entry (trampoline success)
    if(postPc >= 0x80000000 && prePc < 0x80000000
       && !DIAG.loggedAddresses.has('sram')) {
        DIAG.loggedAddresses.add('sram');
        console.log(`[BOOT] SRAM entry cycle=${DIAG.cycleCount}`);
    }

    // First UART byte
    const tx = cpu.read_tx_idx();
    if(tx > 0 && !DIAG.firstUartWrite) {
        DIAG.firstUartWrite = DIAG.cycleCount;
        console.log(`[UART] First TX byte cycle=${DIAG.cycleCount}`);
    }

    // SP sanity every 10k cycles
    if(DIAG.cycleCount % 10000 === 0) {
        const sp = cpu.get_reg(2)>>>0;
```

```
        if(sp && (sp < 0x80000000 || sp > 0x8000FFFF))
            console.error(`[STACK] sp=0x${sp.toString(16)} OUT OF RANGE`);
    }

    // Heartbeat
    if(DIAG.cycleCount % 1000000 === 0)
        console.log(`[HB] cycle=${DIAG.cycleCount}
PC=${(cpu.get_pc()>>>0).toString(16)}`
            + ` uart=${cpu.read_tx_idx()} irq=${cpu.get_irq_count()}`);
}
```

7.4 The Unknown MMIO Log

The single most powerful diagnostic for porting to new samples. Log all dbus accesses to unmapped addresses:

```
// In C++ driver — size 256 for safety:
struct AccessLog { uint32_t addr; uint32_t we; uint32_t val; uint32_t cycle; };
AccessLog unknown_log[256];
int unknown_log_idx = 0;

// At end of dbus handler, after all known addresses:
else if(dadr != 0 && unknown_log_idx < 256) {
    unknown_log[unknown_log_idx++] = {dadr, dwe, ddat, sim_cycle};
}
```

This will immediately reveal any peripheral a new sample is trying to use that you haven't implemented yet.

Part 8: Debugging Playbook — Symptom to Root Cause

8.1 PC Stuck Below 0x80000000

Symptom: Heartbeat shows PC increasing slowly (≈0xE5C8 per 1M cycles). No UART output. No milestones.

Cause: ROM trampoline not planted. SERV is executing nop sled from zeroed ROM.

Fix: Call cpu.load_rom(0, 0x80000297) and cpu.load_rom(1, 0x00028067) after loading binary.

8.2 mtvec Wrong After Boot

Symptom: [POST-CSRW] shows mtvec=0x1c instead of 0x8000001c. First trap derails to ROM.

Cause: ibus_ack asserted on first cycle of ibus_cyc, cutting RF write pipeline short.

Fix: Apply the prev_icyc delay — ack only on second cycle ibus_cyc is high.

Diagnosis: Export rcnt, wen1_r, wdata1_r and log cycles 243-290. Look for wdata1_r going to 0 before chunk 15 is written.

8.3 Trap Storm After Hello World

Symptom: 30+ [TRAP] logs in consecutive cycles. All mcause=0x80000007 (timer IRQ). PC bounces between mtvec and arch_cpu_idle.

Cause: wfi in arch_cpu_idle causes illegal instruction fault every time idle thread runs.

Fix: Apply the Kconfig injection + serv_idle.c override.

8.4 Stall at arch_system_halt

Symptom: PC locked at fixed address after trap storm. uart_bytes stops increasing.

Cause: Zephyr's fatal error handler called arch_system_halt after too many unhandled traps.

Fix: Same as 8.3 — prevent the trap storm.

8.5 UART Log Empty at Cycle 100000

Symptom: UART transaction log shows 0 entries at the 100000-cycle dump.

Cause: Either PC never reached ns16550 init code, or LSR returned wrong value causing the driver to spin.

Fix: Check LSR reads return 0x60 in the correct byte lane. Check mtvec is correct before UART init.

8.6 Register Reads Always Zero

Symptom: get_reg() returns 0 for all registers except x0.

Cause: Attempting to read RF RAM before init() zeroes it, or reading outline signals that report stale zero.

Fix: For GPRs, use get_rf_reg(). For internal signals, only use .curr on registered wire<N> types.

8.7 Unknown MMIO Accesses in Log

Symptom: Unknown MMIO log fills with writes to 0x0C002000 through 0x0C00207C.

Cause: PLIC initialization — Zephyr writing interrupt priorities. This is expected and harmless.

Fix: Add a handler that silently accepts all 0x0C000000 range accesses.

8.8 Derailment to Small Address

Symptom: [DERAILMENT] shows PC jumping from 0x80000224 (mret) to 0xE64.

Cause: mtvec was wrong (0x1000001c) when first trap fired. Handler ran in garbage memory. mepc got corrupted. mret returned to corrupted mepc.

Fix: Fix mtvec first. Everything else cascades from there.

8.9 cnt_en and wcnt Always Read 0

Symptom: All cycles in the RF pipeline trace show cnt_en=0 and wcnt=0 despite CPU executing instructions.

Cause: Both cnt_en and wcnt are /*outline*/ signals in CXXRTL. They are evaluated lazily and their .data[0] values are stale after step().

Fix: Use cnt_lsb (registered wire<4>) as a proxy for cnt_en. Use rcnt (registered wire<5>) to compute wcnt = rcnt - 4 in JavaScript.

Part 9: Lessons — How to Be 10x More Efficient

9.1 What Wasted the Most Time

9.1.1 Not Reading the RTL Files First

We spent significant time guessing about SERV's behavior from black-box observation. Every single internal mystery was immediately resolved once we read the Verilog. The correct workflow is:

1. Read serv_rf_ram_if.v before writing any driver
2. Read serv_state.v to understand ibus_cyc timing
3. Read serv_ctrl.v to understand PC computation
4. THEN write the driver

> 💡 **RULE:** Never guess about hardware behavior. Read the RTL. If you don't have the RTL, get it. Everything in SERV is deterministic and derivable from 7 Verilog files totaling ~1000 lines.

9.1.2 Not Having Internal Signal Probes From Day One

The mtvec bug took orders of magnitude longer to find because we were diagnosing from external observations (mtvec reads back wrong) rather than internal state (rcnt, wen1_r, wdata1_r).

The correct approach: on day one, add exports for all critical internal signals. The minimal set that makes every RF pipeline bug immediately obvious:

- rcnt — tells you exactly where the write pipeline is
- wen1_r — tells you if write enable is asserted
- wdata1_r — tells you what data is being shifted in
- cnt_lsb — tells you if the CPU counter is running

With these four signals, the entire mtvec bug would have been diagnosed in one run instead of many sessions.

9.1.3 Confusing CXXRTL Outline vs Registered Signals

We lost significant time probing cnt_en and wcnt, which always read 0 because they are outline signals. The rule is simple: if the declaration contains /*outline*/, it is stale. Only use .curr on wire<N> types.

9.1.4 Fixing Symptoms Instead of Causes

Several fixes were attempted before the root cause was known: prev_icyc rising-edge fix (correct but not sufficient), set_mtvec manual patch (workaround). The correct approach is to always trace to root cause before fixing.

9.2 The Correct Debugging Protocol

For any new bug, always in this order:

5. Identify the first observable symptom (wrong register value, wrong PC, stall, derailment)
6. Identify the earliest cycle where it first appears (use binary search on cycle counts)
7. Add internal signal probes around that cycle range
8. Run one pass — the trace will make the cause obvious
9. Confirm root cause from RTL before applying any fix
10. Apply the minimal fix that addresses root cause, not symptom
11. Verify with [PRE-/POST-] log pairs

9.3 The Minimal Diagnostic Set for Any New CXXRTL Driver

Before running a single cycle of a new CXXRTL simulation, implement these in order:

12. Heartbeat every 1M cycles (PC, uart_bytes, mtime, irq_count)
13. Derailment detection (PC leaves valid memory range)
14. Stall detection (PC unchanged for 10000 cycles)
15. UART first byte (confirms OS reached print code)
16. Unknown MMIO log (reveals unimplemented peripherals)
17. mtvec check after boot (confirms CSR write pipeline working)
18. Internal RF pipeline exports (rcnt, wen1_r, wdata1_r, cnt_lsb)

Items 1-5 catch 90% of issues. Items 6-7 are specific to SERV. If you have all 7 from the start, most bugs self-diagnose in one run.

9.4 Things That Were Genuinely Not Our Fault

9.4.1 The 1-Cycle ibus_ack Timing Requirement

🔴 **UNDOCUMENTED:** There is no documentation anywhere — not in the SERV README, not in the CXXRTL documentation, not in any SERV example driver — that states ibus_ack must be delayed one cycle for the RF write pipeline to complete. This is a subtle interaction between the RF interface counter (rcnt) and the CPU execution counter (cnt_lsb) that only manifests for CSR write instructions. Any driver that asserts ibus_ack on the first cycle ibus_cyc goes high will silently corrupt CSR register writes.

9.4.2 CXXRTL Outline Signal Staleness

🔴 **UNDOCUMENTED:** CXXRTL documentation does not clearly state that /*outline*/ signals are lazily evaluated and will return stale values when read via .data[0] after step(). This caused significant confusion during the RF pipeline diagnosis. Any future CXXRTL driver writer will hit this.

9.4.3 Zephyr ARCH_HAS_CUSTOM_CPU_IDLE Not Settable in prj.conf

🔴 **MISLEADING:** The Kconfig error message for non-user-settable symbols points to generic documentation that does not explain how to legally select the symbol for RISC-V.

The actual solution (application Kconfig with mainmenu + select) is used by multiple SoCs in the Zephyr tree but not documented as a user-facing pattern.

9.4.4 wfi in SMP Secondary Core Loop

📌 **NOTE:** After fixing arch_cpu_idle, one wfi remains in loop_unconfigured_cores (reset.S:126). This is expected, harmless, and unreachable for single-core SERV. It will always show as 1 remaining wfi in the binary. Do not waste time trying to eliminate it.

Part 10: Quick Reference

10.1 Checklist — Before Running Any Binary

19. ROM trampoline planted (0x80000297, 0x00028067 at ROM words 0, 1)
20. ibus_ack only on SECOND cycle ibus_cyc high (prev_icyc guard)
21. LSR reads return 0x60 in correct byte lane
22. mtimecmp initialized to 0xFFFFFFFF (prevents spurious early IRQ)
23. tick_mtime() called every step()
24. timer_irq updated based on mtime >= mtimecmp
25. dbus_ack asserted every cycle dbus_cyc is high
26. ibus_ack and dbus_ack cleared after rising edge
27. RF RAM zeroed in init() via 576-entry loop
28. 200 reset cycles with rst_n=0 before releasing reset

10.2 Checklist — After First Boot

29. cycle ~105: [BOOT] SRAM entry logged
30. cycle ~285: mtvec=0x8000001c confirmed
31. cycle ~353: __reset reached
32. cycle ~421: __initialize reached
33. cycle ~712916: .text section start (Zephyr kernel running)
34. cycle ~755118: First UART TX byte
35. UART log shows 8 transactions (ns16550 init sequence)
36. No derailment before first UART byte

10.3 Signal Name Cheat Sheet

Signal	CXXRTL Name
rcnt	p_cpu_2e_rf__ram__if_2e_rcnt
wen1_r	p_cpu_2e_rf__ram__if_2e_wen1__r
wdata1_r	p_cpu_2e_rf__ram__if_2e_wdata1__r
rtrig1	p_cpu_2e_rf__ram__if_2e_rtrig1
rgnt	p_cpu_2e_rf__ram__if_2e_rgnt
rreq_r	p_cpu_2e_rf__ram__if_2e_rreq__r
cnt_lsb	p_cpu_2e_cpu_2e_state_2e_gen__cnt__w__eq__1_2e_cnt__lsb
wcnt	p_cpu_2e_rf__ram__if_2e_wcnt
cnt_en	p_cpu_2e_cpu_2e_state_2e_o__cnt__en
o_wen1	p_cpu_2e_cpu_2e_rf__if_2e_o__wen1

10.4 Timing Quick Reference

Event	Cycle (relative to fetch)
ibus_cyc first cycle high	0
ibus_ack asserted (with fix)	+1
i_rreq fires (ibus_ack → rf_ram_if)	+1
rreq_r = 1	+1
rgnt = 1	+2
cnt_lsb gets 1 (CPU counter starts)	+2
cnt_en = 1	+2
First wtrig1 (wcnt=1, rcnt=5)	+5
wen1_r latched from previous value	+5 (stale — must pre-set from prior instruction)
First actual CSR write (uses wen1_r.curr)	+7 (rcnt=7, wcnt=3)
Last CSR write (chunk 15)	+35 (rcnt wraps to 3, wcnt=31)
Next ibus_ack resets rcnt	+37

Appendix: Files Modified in Zephyr Tree

For reproducibility, these are the ONLY files that need to be created or modified (nothing in arch/, soc/, or boards/ is touched):

File	Action
`samples/hello_world/Kconfig`	CREATE
`samples/hello_world/src/serv_idle.c`	CREATE
`samples/hello_world/CMakeLists.txt`	APPEND

✅ **CLEAN:** The Zephyr OS source tree (arch/, soc/, boards/, kernel/, drivers/) is completely unmodified. west update will not cause conflicts. The build is reproducible from any clean Zephyr checkout.

End of Document